

Tiles for mapping

Most of my effort so far has been directed at understanding objects built from points, such as state borders or rivers.

Fundamentally different are basemaps, which are basically images. These can vary in extent and in the level of detail they show. A single file with a given bounds and zoom level is called a **tile**.

Tiles can be obtained from a tile server by constructing a url like `https://tile.openstreetmap.org/{z}/{x}/{y}.png`, where `{z}` is replaced with an integer in an appropriate range, and the same for `{x}` and `{y}`.

`z` is the zoom level, `x` and `y` are indexes for the tile being requested.

The first zoom level is `z=0`, the whole world. The url was `https://tile.openstreetmap.org/0/0/0.png`

Here is a link: [world](#)

The image as I have it cached in `../figs/0_0_0.png`



There are 2^n tiles in each row and column for zoom `z = n`.

The next layer, with `z = 1` has four tiles. Here are some links:

- [1/0/0](#) is north of the equator and west of Greenwich.
- [1/1/1](#) is south of the equator and east of Greenwich.

A cached image of the first one `../figs/1_0_0.png` :



We can see from this that the zoom level starts with `0` (as usual in Python), and the indexing of rows and columns also starts with `0`.

At zoom level `n`, there should be 2^n tiles in each row and column. So if zoom = 10, that's $2^{10} = 1024$. Each tile is about $360/1024$ 0.35 degrees of longitude and $180/1024$ or half that of latitude? Since the images are square, they are stretched vertically.

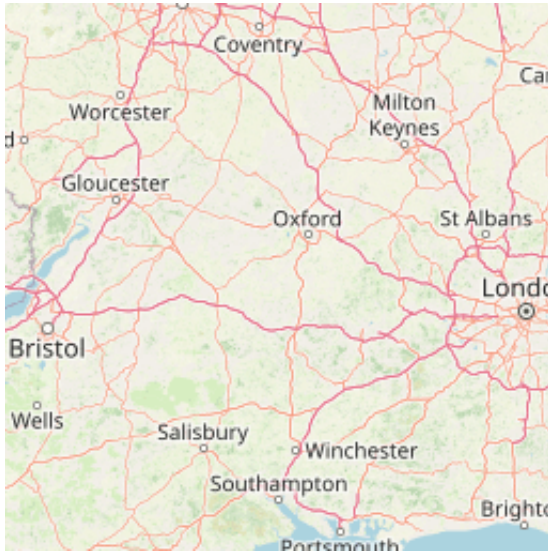
<https://wiki.openstreetmap.org/wiki/Tiles> has an example.

<https://tile.openstreetmap.org/7/63/42.png>

[7/63/42](#)

I have cached the image as `../figs/7_63_42.png`.

This is zoom level 7 with tile number `63/42`.



What we would expect then, is that the tile obtained with `8/126/84` should be the NW quarter of tile `7/63/42`. And you can check that it is:

[8/126/84](#)

Calculating the tile number for a given place.

The coordinates we're using come from the Mercator projection of the world as LAT,LON (from say, Google Maps) or LON,LAT (from say, GeoPandas).

The tiles in any row extend from `LON = [-180, 180]`, centered on Greenwich. Starting with a LON, we should add 180 to get a number in the range `[0, 360]`, then calculate what fraction of `360` that is. Finally we find the tile number by applying that fraction to `2**z`, the number of tiles in the complete row.

```
tile_x = (LON+180)/360 * 2**z
```

This works.

Kiawah Island, which is just to the SW of Folly, contains `LON = -80.0000`. Starting with zoom level 6, there should be `2^6` or 64 tiles. We are one quarter of the way across the grid, so we request

[6/17/32](#)



Near Guyaquil, on the equator. Working our way north, we proceed through

- SE -> <https://tile.openstreetmap.org/7/35/51.png>
- SE -> <https://tile.openstreetmap.org/8/71/103.png>
- NW -> <https://tile.openstreetmap.org/9/142/206.png>
- SW -> <https://tile.openstreetmap.org/10/284/413.png>
- SE -> <https://tile.openstreetmap.org/11/569/827.png> (Folly Beach)
- NW -> <https://tile.openstreetmap.org/12/1138/1654.png>

and eventually reach

[13/2276/3309](https://tile.openstreetmap.org/13/2276/3309)

Folly Beach is $-79.94, 32.65$. And if you do the math, you'll find

```
In [1... import math

def tile_x(z, LON):
    f = (LON+180)/360
    return math.floor(f * 2**z)

tile_x(13, -79.94)
```

Out [1... 2276

However, a similar approach to LAT and what we will call `tile_y` fails. Before I show how that works, let's go through an example to obtain the

correct tile_x and tile_y for Folly Beach, SC.

The explanation is here:

https://wiki.openstreetmap.org/wiki/Slippy_map_tilenames

We should

- convert LAT in degrees to radians
- find the tangent and the secant of that
- find the natural logarithm of that and divide by pi
- subtract the result from 1
- multiply by 2^{z-1}

```
In [2... from math import *

def tile_y(z, LAT):
    rad = radians(LAT)
    s = tan(rad) + 1/cos(rad)
    d = 1 - log(s)/pi
    return math.floor(d*2**(z-1))

tile_y(13, 32.65)
```

Out [2... 3309

I couldn't help feeling that this formula is related somehow to

$$\int \sec \theta \, d\theta = \ln |\sec \theta + \tan \theta|$$

Mercator projection

If you look at the map in the [wikipedia](#) article on the Mercator projection (or the tiles for zoom = 0 and 1 above, you will be reminded that the projection gets very wonky at you get closer to the poles.

It's unusable above latitude 85 degrees or so.

The description of the projection is quite simple. We take an idealized earth (spherical) and surround it with a cylinder, touching at the equator. Any point (LON, LAT) on the surfact of the sphere is projected onto

the cylinder by forming a line from the center of the earth through the point and extending it to the cylinder.